# hash-turbo User Manual

> Version 1.0.1

Cross-platform file hash management tool with CLI and PySide6 QML GUI. Generates, stores, verifies, and sanitizes file hashes with support for many algorithms.

---

## Table of Contents


- Quick Start
- Supported Algorithms
- GUI
- Hash View
- Verify View
- Sanitize View
- Settings
- CLI
- hash — Generate Hashes
- verify — Verify Hashes
- sanitize — Transform Hash Files
- algorithms — List Algorithms
- Hash File Formats
- Distribution


---

## Quick Start

```
# Hash a file
hash-turbo hash report.pdf

# Hash a directory recursively, write to a file
hash-turbo hash src/ -r -o checksums.sha256

# Verify files against a hash file
hash-turbo verify checksums.sha256
```

```
# Launch the GUI
hash-turbo gui
# Or just:
hash-turbo
```

---

## Supported Algorithms

hash-turbo supports all major hash algorithms via Python's `hashlib`:

| Algorithm | CLI flag | Notes |
|---|---|---|
| MD5 | `md5` | Fast, not collision-resistant |
| SHA-1 | `sha1` | Legacy, not recommended for security |
| SHA-224 | `sha224` | Truncated SHA-256 |
| SHA-256 | `sha256` | **Default** — widely used, recommended |
| SHA-384 | `sha384` | Truncated SHA-512 |
| SHA-512 | `sha512` | Stronger variant of SHA-2 |
| SHA3-256 | `sha3-256` | SHA-3 family |
| SHA3-512 | `sha3-512` | SHA-3 family |
| BLAKE2b | `blake2b` | Fast, modern |
| BLAKE2s | `blake2s` | Fast, optimized for 32-bit |

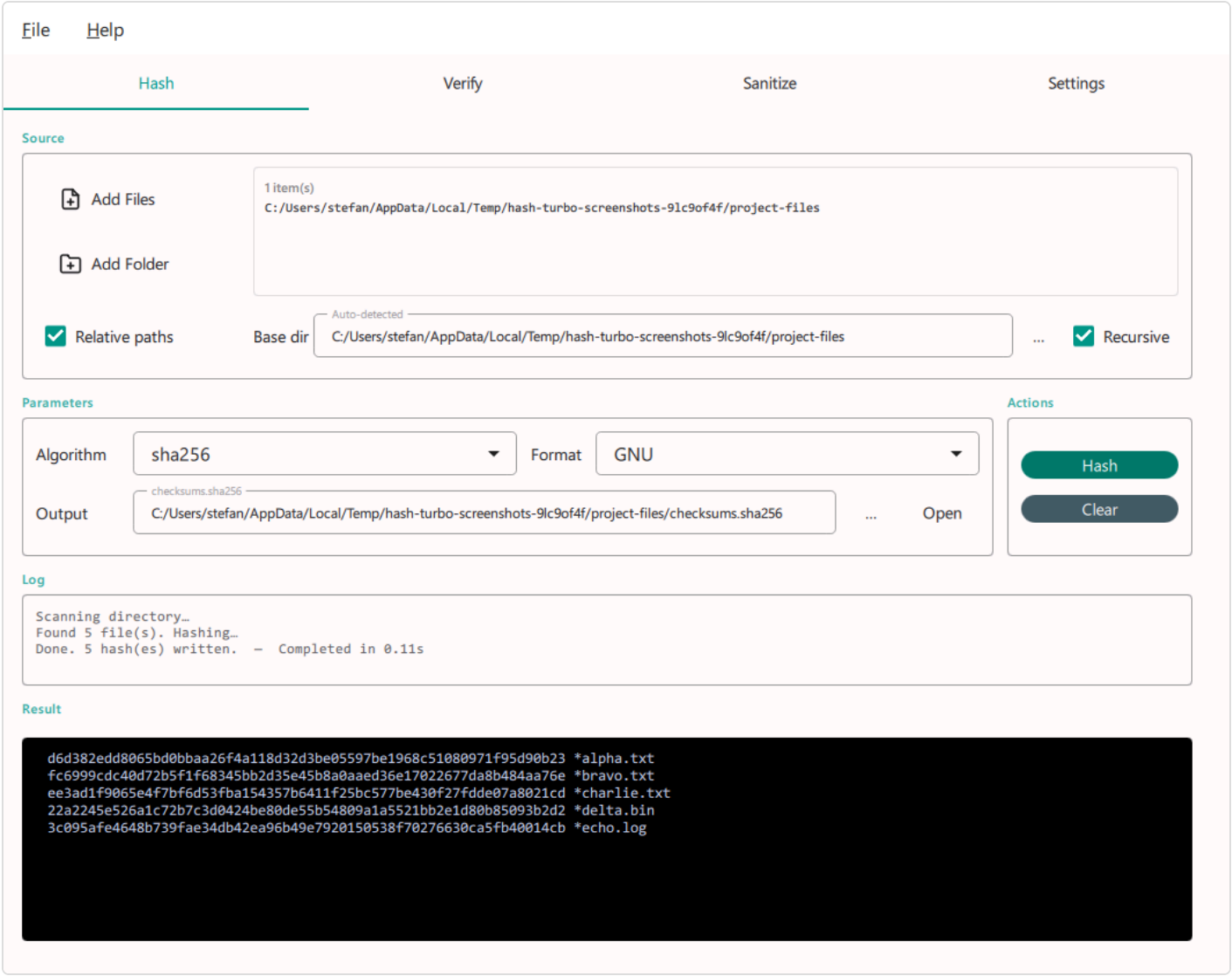
Run `hash-turbo algorithms` to list all available algorithms at runtime.

---

## GUI

The GUI launches automatically when you run `hash-turbo` without a subcommand, or explicitly with `hash-turbo gui`. It uses a Material Design theme with four tabs: **Hash**, **Verify**, **Sanitize**, and **Settings**.

### Hash View




The Hash view generates file hashes and writes them to a hash file.

### Source

Add files to hash using any of these methods:

- **Add Files** button — opens a file picker for individual files
- **Add Folder** button — opens a folder picker to add an entire directory
- **Drag and drop** — drop files or folders onto the drop zone

The drop zone displays the number of pending items and lists their paths. When you add a folder, the **Base dir** field auto-fills with the folder path.

#### Options

- **Relative paths** checkbox — when checked, paths in the output hash file are relative to the base directory (default: checked)

- **Base dir** field — the root directory for computing relative paths; auto-detected from the first added folder
- **Recursive** checkbox — scan subdirectories (default: checked)

**Parameters**

- **Algorithm** — select the hash algorithm from the dropdown (default: SHA-256, configurable in Settings)
- **Format** — choose between GNU and BSD output format (default: GNU, configurable in Settings)
- **Output** — path to the output hash file; auto-filled as `<base_dir>/checksums.<algorithm>` when a folder is added. Click **...** to browse, or **Open** to open the generated file after hashing.

**Actions**

- **Hash** — start hashing all pending files. The button changes to **Cancel** while hashing is in progress.
- **Clear** — remove all pending files and reset the form.

**Log**

Displays real-time progress during hashing:

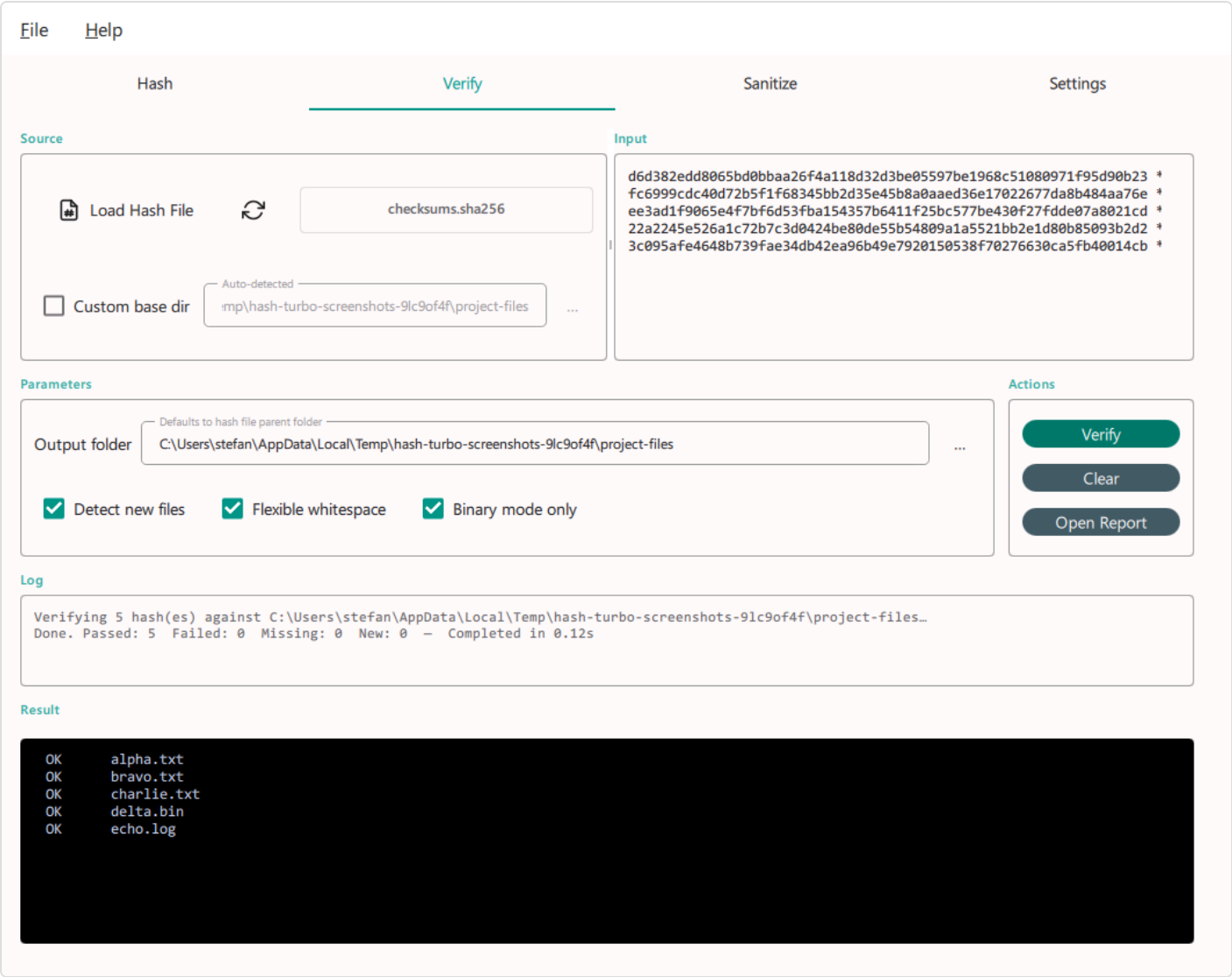
- A progress bar with percentage and current file name
- Status messages: scanning progress, file count, completion summary
- Final line includes elapsed time (e.g. `Done. 5 hash(es) written. — Completed in 1.23s`)

**Result**

A monospace terminal panel (black background) showing the generated hash entries as they are computed. Displays the last 200 lines in a rolling view. Output follows the selected format (GNU or BSD).

---

## Verify View




The Verify view checks file integrity by comparing computed hashes against expected values from a hash file.

### Source

Load a hash file using any of these methods:

- **Load Hash File** button — opens a file picker (filters: `.sha256`, `.md5`, `.sha1`, `.sha512`)
- **Drag and drop** — drop a hash file onto the drop zone
- **Reload** button (appears after loading) — re-reads the file from disk

The loaded file name is displayed in the drop zone and can be clicked to open it in the default application.

#### Options

- **Custom base dir** checkbox — override the auto-detected base directory for resolving file paths
- Base directory field — where to look for the files listed in the hash file (defaults to the hash file's parent directory)

#### Input

A read-only monospace text area showing the content of the loaded hash file. For large files (200+ entries), only a preview is shown with an entry count indicator.

#### Parameters

- **Output folder** — directory for verification report output (defaults to the hash file's parent)
- **Detect new files** — scan for files not listed in the hash file (default: checked)
- **Flexible whitespace** — accept tabs and multiple spaces in GNU format entries (default: checked)
- **Binary mode only** — always hash in binary mode, ignoring the text/binary indicator (default: checked)

#### Actions

- **Verify** — start verification. Changes to **Cancel** while running.
- **Clear** — reset all fields and results.
- **Open Report** — open the generated verification report file (enabled after verification completes).

#### Log

Shows verification progress with:

- A progress bar with percentage and current file name
- Status messages for each file being verified
- Summary line: `Passed: N Failed: N Missing: N New: N — Completed in X.XXs`

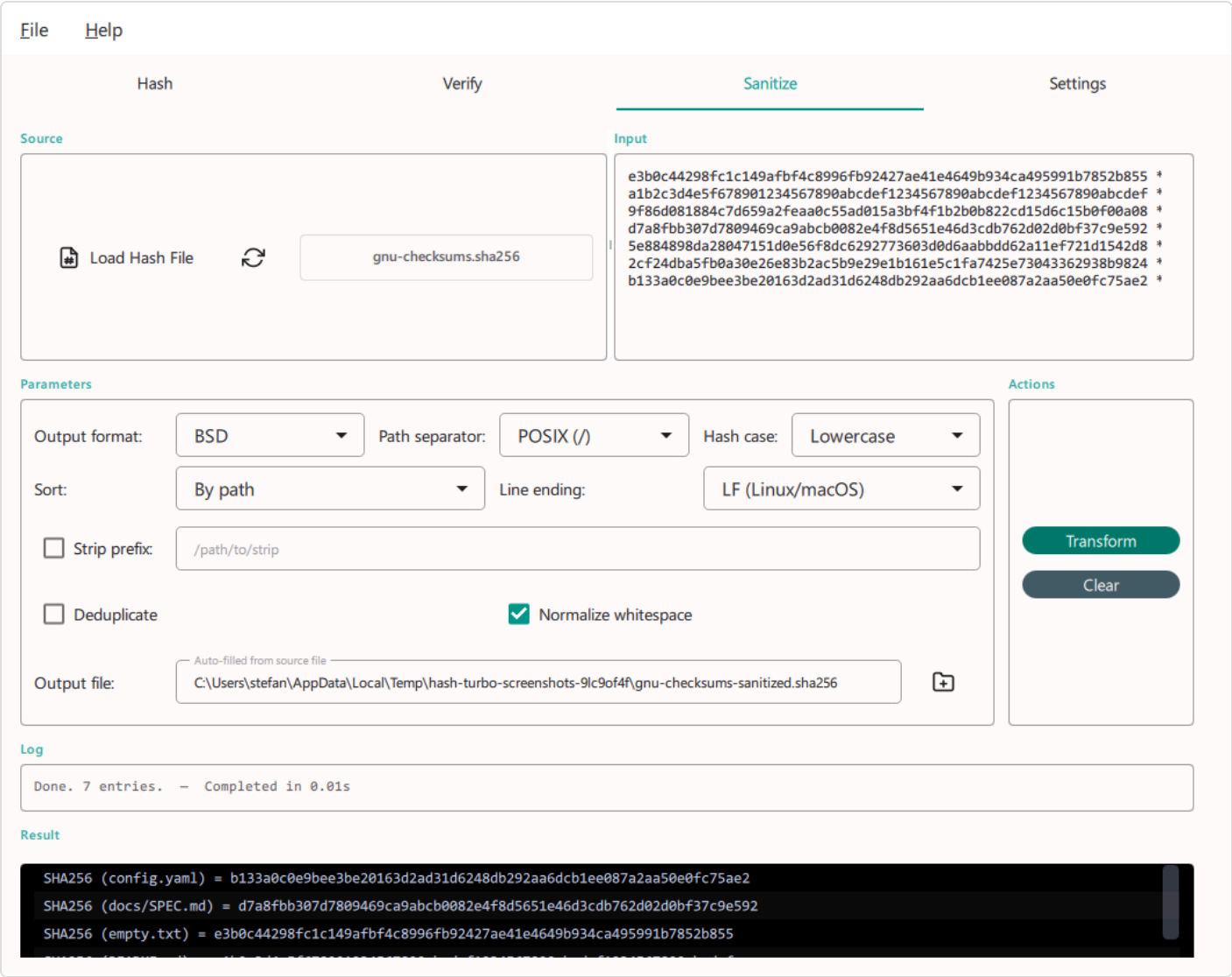
#### Result

A monospace terminal panel showing per-file results with status indicators:

- `OK` — hash matches
- `FAIL` — hash mismatch (shows expected vs. computed)
- `MISSING` — file not found on disk
- `NEW` — file exists but is not listed in the hash file (when "Detect new files" is enabled)

---

## Sanitize View




The Sanitize view transforms hash manifest files — converting between formats, normalizing paths, and cleaning up entries.

### Source

Load a hash file using any of these methods:

- **Load Hash File** button — opens a file picker
- **Drag and drop** — drop a hash file onto the drop zone
- **Reload** button (appears after loading) — re-reads the file from disk

### Input

A read-only monospace text area showing the loaded hash file content. For large files (200+ entries), only a preview is shown.

### Parameters

| Parameter | Options | Description |
|---|---|---|
| **Output format** | GNU, BSD | Target format for the transformed output |
| **Path separator** | Keep original, POSIX ( `/` ), Windows ( `\` ) | Normalize path separators in file paths |
| **Hash case** | Keep original, Lowercase, Uppercase | Normalize hex digest casing |
| **Sort** | None, By path, By hash, Filesystem | Sort entries by the selected key |
| **Line ending** | System, LF, CRLF, CR | Control line ending style in output |
| **Strip prefix** checkbox + field | Text | Remove a leading path prefix from all entries |
| **Deduplicate** checkbox | On/Off | Remove duplicate entries by path |
| **Normalize whitespace** checkbox | On/Off | Fix irregular whitespace in GNU format (default: checked) |

#### Output file

- Auto-filled as `<stem>—sanitized.<ext>` when a file is loaded
- Click **...** to browse for a custom output path
- Click the file icon to open the output file after transformation

### Actions

- **Transform** — apply all selected transformations. Changes to **Cancel** while running.
- **Clear** — reset all fields and results.

### Log

Displays status messages after each transformation:

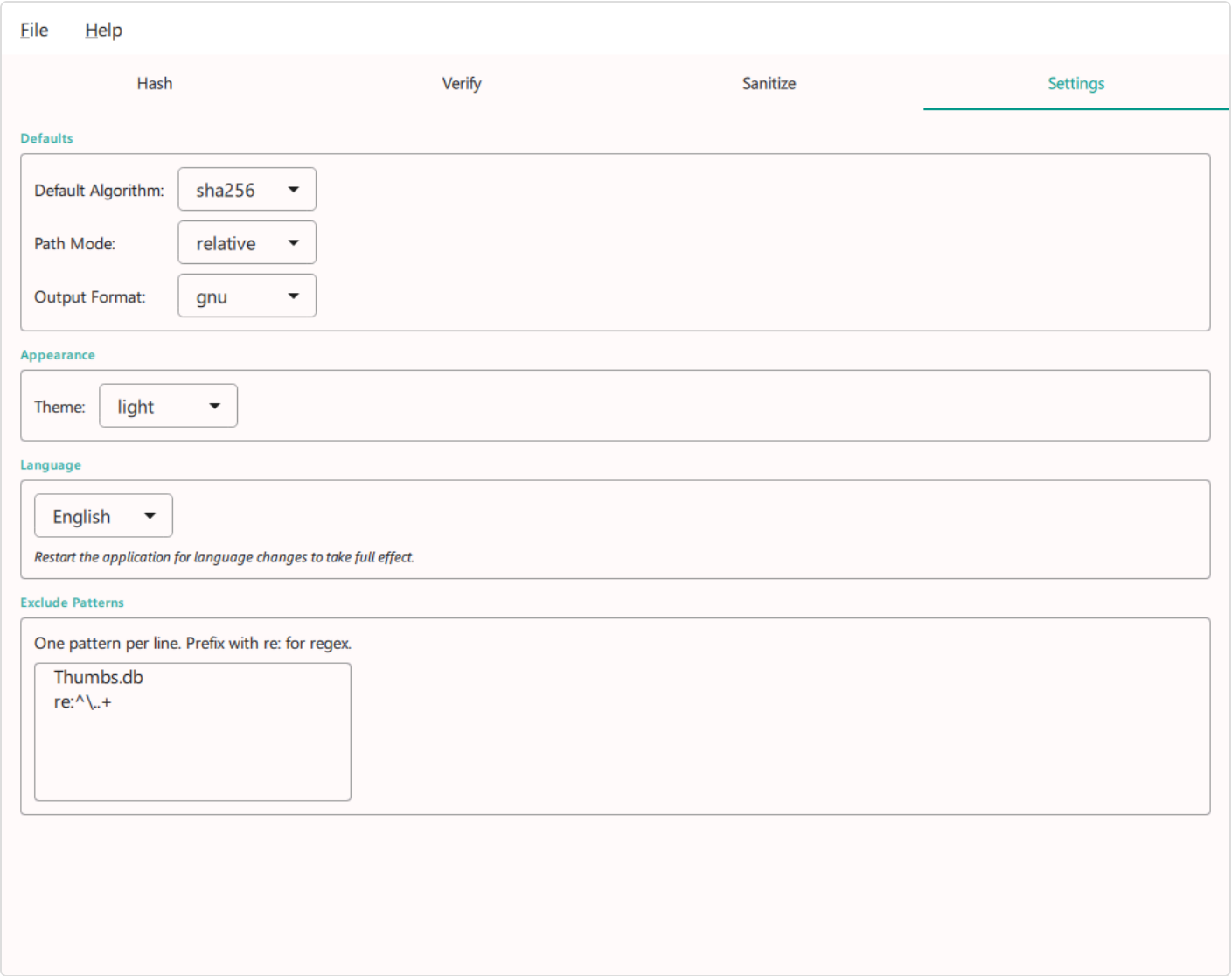
- Completion summary with entry count and elapsed time (e.g. `Done. 7 entries. — Completed in 0.05s` )
- Error messages if the input cannot be parsed

### Result

A monospace terminal panel (black background) showing the transformed hash entries in a list view. Rows have alternating backgrounds for readability. Output is auto-saved to the configured output file.

---

## Settings




The Settings view configures application defaults. All settings are persisted across sessions.

### Defaults

- **Default Algorithm** — the hash algorithm pre-selected in the Hash view (default: SHA-256)
- **Path Mode** — relative or absolute paths in hash output (default: relative)
- **Output Format** — GNU, BSD, or JSON (default: GNU)

### Appearance

- **Theme** — System (follows OS), Light, or Dark

### Language

- **Language selector** — English, Deutsch, Français, Italiano, Rumantsch
- Changes require an application restart to take full effect
- Translations powered by gettext ( `.po` / `.mo` files)

### Exclude Patterns

Define filename patterns to skip during hashing and verification:

- One pattern per line
- Supports fnmatch glob syntax (e.g., `Thumbs.db`, `*.tmp`)
- Prefix with `re:` for regex patterns (e.g., `re:^\..+` to skip hidden files)
- Defaults: `Thumbs.db`, `re:^\..+`

---

## CLI

The CLI provides the same core features as the GUI, optimized for scripting and pipelines.

### hash — Generate Hashes

```
hash-turbo hash <paths ... > [options]
```

Generate hashes for one or more files or directories.

#### Arguments

| Argument | Description |
|---|---|
| `<paths ... >` | One or more file or directory paths (required) |

#### Options

| Option | Description | Default |
|---|---|---|
| `-a`, `--algorithm <algo>` | Hash algorithm | `sha256` |
| `-r`, `--recursive` | Recurse into directories | Off |
| `-g`, `--glob <pattern>` | Filter files by glob pattern | None |
| `--exclude <pattern>` | Exclude files matching pattern | None |
| `-o`, `--output <file>` | Write hashes to a file | stdout |
| `--format {gnu,bsd,json}` | Output format | `gnu` |
| `--path-mode {relative,absolute}` | Path style in output | `relative` |
| `--base-dir <dir>` | Base directory for relative paths | hash file parent |
| `-j`, `--jobs <n>` | Number of parallel workers | CPU count |

| Option | Description | Default |
|---|---|---|
| `-v` , `--verbose` | Verbose output | Off |
| `-q` , `--quiet` | Quiet output | Off |

**Examples**

```
# Hash a single file (friendly display)
hash-turbo hash report.pdf
# → SHA-256: 3a7bd3e2 ... af91c  report.pdf

# Hash a directory, write GNU format
hash-turbo hash src/ -r -o checksums.sha256

# BSD format with MD5
hash-turbo hash *.zip -a md5 --format bsd

# JSON output
hash-turbo hash build/ -r --format json -o hashes.json

# Parallel hashing with 8 workers
hash-turbo hash data/ -r -j 8 -o checksums.sha256

# Only Python files, excluding tests
hash-turbo hash src/ -r -g "*.py" --exclude "test_*"
```

---

## verify — Verify Hashes

```
hash-turbo verify <hashfile> [options]
hash-turbo verify <file> --expect <hash> [-a <algo>]
```

Verify files against a hash file, or verify a single file against an expected hash.

**Options**

| Option | Description | Default |
|---|---|---|
| `--expect <hash>` | Verify a single file against this hash | None |
| `--strict` | Fail on missing files | Off |
| `-a` , `--algorithm <algo>` | Algorithm for `--expect` mode | `sha256` |

| Option | Description | Default |
|---|---|---|
| `--flexible-whitespace` / `--no-flexible-whitespace` | Accept irregular whitespace in GNU format | On |
| `--binary-only` / `--no-binary-only` | Always hash in binary mode | On |
| `-v` , `--verbose` | Show expected/computed hashes on failure | Off |
| `-q` , `--quiet` | Only show failures | Off |

**Exit Codes**

| Code | Meaning |
|---|---|
| `0` | All files verified OK |
| `1` | One or more mismatches or missing files (with `--strict` ) |
| `2` | Usage error (no hash file, no entries found) |

**Examples**

```
# Verify all files in a hash file
hash-turbo verify checksums.sha256

# Strict mode — fail on missing files
hash-turbo verify checksums.sha256 --strict

# Quiet mode — only show failures
hash-turbo verify checksums.sha256 -q

# Inline verification of a single file
hash-turbo verify download.zip --expect 3a7bd3e2360a3d29eea436fcfb7e44c735d117c42d1c1835420b6b9942

# Verbose — show expected vs computed on failure
hash-turbo verify checksums.sha256 -v
```

---

## sanitize — Transform Hash Files

```
hash-turbo sanitize <hashfile> [options]
```

Transform a hash manifest file: convert formats, normalize paths, deduplicate, and sort.

**Options**

| Option | Description | Default |
|---|---|---|
| `--format {gnu,bsd}` | Output format | Auto-detect from input |
| `--separator {keep,posix,windows}` | Path separator style | `keep` |
| `--strip-prefix <path>` | Remove leading path prefix | None |
| `--hash-case {keep,lower,upper}` | Hex digest casing | `keep` |
| `--sort {none,path,hash,filesystem}` | Sort entries | `none` |
| `--deduplicate` | Remove duplicate entries by path | Off |
| `--line-ending {system,lf,crlf,cr}` | Line ending style | `system` |
| `--normalize-whitespace` / `--no-normalize-whitespace` | Fix irregular whitespace | On |
| `-o`, `--output <file>` | Write result to file | stdout |

**Examples**

```
# Convert GNU to BSD format
hash-turbo sanitize checksums.sha256 --format bsd

# Normalize to POSIX paths with lowercase hashes
hash-turbo sanitize checksums.sha256 --separator posix --hash-case lower

# Strip a path prefix to make entries relative
hash-turbo sanitize checksums.sha256 --strip-prefix C:/projects/myapp

# Deduplicate and sort, write to a new file
hash-turbo sanitize checksums.sha256 --deduplicate --sort path -o cleaned.sha256

# Full cleanup pipeline
hash-turbo sanitize checksums.sha256 \
    --format bsd \
    --separator posix \
    --hash-case lower \
    --sort path \
    --deduplicate \
    --line-ending lf \
    -o normalized.sha256
```

### algorithms — List Algorithms

```
hash-turbo algorithms
```

Lists all available hash algorithms at runtime. Includes all algorithms supported by the platform's `hashlib`.

---

## Hash File Formats

hash-turbo supports two standard hash file formats and JSON:

### GNU Format

The default format, compatible with `sha256sum`, `md5sum`, etc.

```
3a7bd3e2360a3d29eea436fcfb7e44c735d117c42d1c1835420b6b9942dd4f1b *src/main.py
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 *README.md
```

- `<hex_digest> *<path>` (binary mode) or `<hex_digest> <path>` (text mode)
- The `*` prefix indicates binary mode

### BSD Format

```
SHA256 (src/main.py) = 3a7bd3e2360a3d29eea436fcfb7e44c735d117c42d1c1835420b6b9942dd4f1b
SHA256 (README.md) = e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

- `<ALGORITHM> (<path>) = <hex_digest>`

### JSON Format

Available for CLI output only (`--format json`):

```
[
  {
    "path": "src/main.py",
    "algorithm": "sha256",
    "hash": "3a7bd3e2 ... "
  }
]
```

### Auto-Detection

hash-turbo automatically detects the format when reading hash files. Both GNU and BSD formats are recognized regardless of file extension.

---

## Distribution

hash-turbo is distributed as two standalone executables built with PyInstaller:

| Binary | Console | Purpose |
|---|---|---|
| `hash-turbo` | Yes | CLI usage — full terminal stdin/stdout/stderr support |
| `hash-turbo-gui` | No | GUI — launches without a console window |

Both binaries bundle the complete Python runtime and all dependencies. No installation required — just download and run.